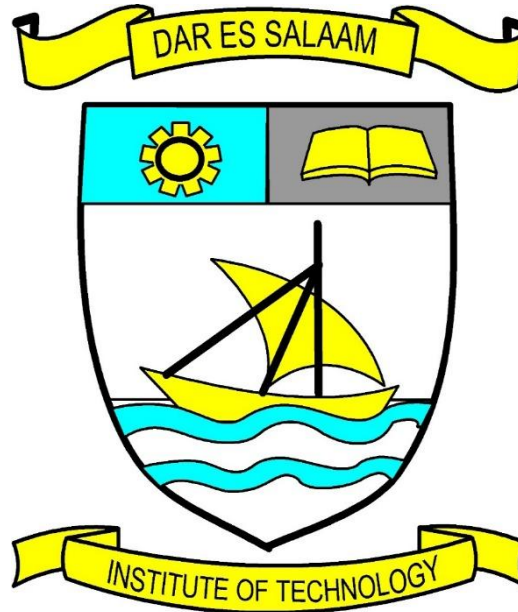


DAR ES SALAAM INSTITUTE OF TECHNOLOGY



TECHNICAL PROJECT REPORT

ONLINE VOTING SYSTEM

A Secure Web-Based Platform for Conducting Electronic Elections

CLASS: BENG23COE-02

DEPARTMENT: COMPUTER DEPARTMENT

GROUP: GROUP 4

MODULE: DATABASE PROGRAMMING AND ADMINISTRATION (COU 07502)

MEMBERS

	NAMES	REGISTRATION NO
1	EWARD EDWARD LUTEMA	230242454635
2	EUNICE MONICA KAHELO	230242417582
3	GERMANUS YENS NDUNGURU	230242427755
4	EDDY VICTOR MWAKALINGA	230242437523
5	NELSON FELICHISM SHAYO	230242467611

TABLE OF CONTENT

ABSTRACT	4
LIST OF TABLES	5
LIST OF FIGURES	6
ABBREVIATIONS AND ACRONYMS.....	7
CHAPTER ONE	8
INTRODUCTION	8
1.1 Background of the Study	8
1.2 Problem Statement	8
1.3 Purpose of the Project.....	8
1.4 Objectives of the Project	9
Specific objective	9
1.5 Scope of the Project	9
1.6 Significance of the Study	9
CHAPTER TWO.....	10
2.1 LITERATURE REVIEW	10
CHAPTER THREE	11
3.1 METHODOLOGY	11
WATERFALL MODEL IMPLEMENTATION	11
CHAPTER FOUR.....	14
4.1 SYSTEM DESIGN	14
4.1.1 SYSTEM ARCHITECTURE.....	14
4.1.2 DATABASE STRUCUTURE:.....	14
ERD DIAGRAM	17
RELATIONSHIPS BETWEEN ENTITIES	17
USE CASE DIAGRAM.....	18
TECHNOLOGY USED	22
3.1.2 SYSTEM ARCHITECTURE.....	22
CHAPTER FIVE.....	23
IMPLEMENTATION	23
CHAPTER FIVE.....	33
5.1 TESTING AND RESULTS	33

5.2 CONCLUSION AND REMMENDATIONS	35
REFERENCES	36
APPENDICES	37

ABSTRACT

The Online Voting System is a web-based application developed to provide a secure, efficient, and reliable method of conducting elections electronically. Traditional voting systems rely heavily on manual processes such as printing ballot papers, physical voter presence, manual counting, and result tabulation, which often lead to delays, increased costs, and potential human errors. These limitations create the need for a modern solution that can streamline electoral processes while maintaining high levels of security and transparency.

This project addresses these challenges by designing and implementing a digital voting platform that allows users to register, authenticate securely, and cast votes remotely through an intuitive web interface. The system incorporates role-based access control, distinguishing between administrators and voters. Administrators manage candidate records, monitor election activities, and view results, while registered voters can securely participate in elections.

The system was developed using PHP for backend processing and Oracle Database for data storage and management. Security features such as password hashing, session management, database constraints, and triggers were implemented to ensure data integrity, prevent unauthorized access, and eliminate duplicate voting. The final system successfully demonstrates real-time vote processing, efficient result generation, and a user-friendly interface that can be accessed across multiple devices.

LIST OF TABLES

Table 1: System Requirements Summary

Table 2: Users Table Structure

Table 3: Candidates Table Structure

Table 4: Votes Table Structure

Table 5: Testing Summary Table

LIST OF FIGURES

LIST OF FIGURES

- Figure 1: User Table Creation Script
- Figure 2: User ID Sequence Creation
- Figure 3: User Table Trigger
- Figure 4: Candidate Table Creation Script
- Figure 5: Candidate Table Trigger
- Figure 6: Votes Table Creation Script
- Figure 7: Votes Table Trigger
- Figure 8: Registration Page Interface
- Figure 9: Login Page Interface
- Figure 10: Voter Dashboard
- Figure 11: Admin Dashboard (Page 1)
- Figure 12: Admin Dashboard (Page 2)
- Figure 13: Admin Dashboard (Page 3)
- Figure 14: Limited Admin Dashboard (Page 1)
- Figure 15: Limited Admin Dashboard (Page 2)
- Figure 16: Duplicate Voting Test Result
- Figure 17: Entity Relationship Diagram (ERD)
- Figure 18: Use Case Diagram

ABBREVIATIONS AND ACRONYMS

DIT – Dar es Salaam Institute of Technology

ERD – Entity Relationship Diagram

SQL – Structured Query Language

RBAC – Role-Based Access Control

DBMS – Database Management System

PHP – Hypertext Preprocessor

HTML – HyperText Markup Language

CSS – Cascading Style Sheets

JS – JavaScript

UML – Unified Modeling Language

CHAPTER ONE

INTRODUCTION

Voting is a democratic process used by institutions to make collective decisions. Traditional paper-based voting presents challenges such as delays, high costs, and risk of duplicate voting.

1.1 Background of the Study

Voting is a fundamental process used by organizations, institutions, and governments to make collective decisions through democratic participation. Traditionally, voting has been conducted using paper ballots, requiring voters to be physically present at polling stations. While this method has been widely used, it presents several limitations including logistical challenges, high operational costs, slow vote counting processes, and risks of human error.

With advancements in information technology, many sectors have transitioned from manual operations to automated digital systems. Online voting systems have emerged as a modern solution that enables voters to cast ballots electronically, improving efficiency, accuracy, and accessibility. The development of secure web technologies and database systems has made it possible to create reliable platforms that ensure transparency while maintaining voter confidentiality.

1.2 Problem Statement

Traditional voting systems face numerous challenges that hinder their effectiveness. Manual vote counting is time-consuming and prone to human error, often causing delays in announcing election results. The requirement for physical presence limits voter participation, especially for individuals who are unable to travel to polling stations. Additionally, the printing and distribution of ballot papers incur significant costs, and there is always a risk of ballot tampering or misplacement.

Another major issue is the difficulty in preventing duplicate voting, which can compromise election fairness. These challenges highlight the need for a secure, efficient, and automated voting solution that minimizes errors and enhances transparency.

1.3 Purpose of the Project

The purpose of this project is to design and implement a secure web-based voting system that automates the entire electoral process. The system aims to provide a platform where users can register, log in securely, vote electronically, and view election results instantly. It also aims to provide administrators with tools for managing candidates and monitoring election activities.

1.4 Objectives of the Project

General Objective

To develop a secure and efficient Online Voting System that facilitates electronic elections.

Specific objective

- Secure user registration and login
- Create secure sessions for authenticate users
- Provide administrator with a centralized management interface
- Prevent user from voting more than once for the same position
- Display real-time vote counts for all candidates and position

1.5 Scope of the Project

The system focuses on providing essential functionalities required for conducting online elections. It includes user registration, secure login, candidate management, vote casting, and real-time results display. The system is web-based and can be accessed using standard web browsers.

However, advanced features such as biometric authentication, blockchain integration, and mobile application support are beyond the scope of this project.

1.6 Significance of the Study

This project demonstrates the practical application of database management systems, web development technologies, and cybersecurity principles. It provides a real-world solution for institutions seeking to modernize their election processes while also serving as an educational tool for understanding system design, secure authentication, and database implementation.

CHAPTER TWO

2.1 LITERATURE REVIEW

INTRODUCTION

Online voting systems have been created by different organizations to improve how elections are conducted. Platforms such as SimplyVoting and ElectionBuddy provide cloud-based services that allow users to organize and manage elections electronically. These systems offer convenience and automation, making the voting process faster and easier. However, they usually require subscription payments and store sensitive data on external servers, which can raise concerns about privacy and data security.

Modern online voting systems are built using various technologies. Common tools include web programming languages such as PHP, relational database systems like Oracle, and security mechanisms such as Crypt for password hashing. These technologies form the foundation for developing secure, efficient, and scalable applications. Even though these systems have clear benefits, many of them do not offer enough customization options. This makes them unsuitable for institutions with specific needs. In addition, some systems have complicated user interfaces that may not be easy for everyone to use. This project aims to solve these problems by developing a cost-effective, customizable, and locally hosted voting system.

Electronic voting (e-voting) systems have greatly changed traditional election processes. They address major issues such as security, public acceptance, and convenience. Compared to paper-based voting methods, which often involve high costs, slow vote counting, logistical difficulties, and human errors, e-voting systems promise improved efficiency, accuracy, and accessibility. Over time, these systems have continued to develop, although acceptance levels differ from one country to another depending on various social, political, and technological factors.

There are different types of electronic voting systems. Direct Recording Electronic (DRE) systems are among the earliest forms. They use electronic machines with touchscreens or buttons to record votes directly into computer memory. While they remove the need for paper ballots and speed up the process, they have been criticized for lacking physical audit trails and being vulnerable to software manipulation.

Internet voting systems allow voters to cast their ballots online from any location with internet access. This method increases convenience and can improve voter participation by removing geographical barriers. For example, online voting has been shown to increase participation by about 3.5 percentage points.

Blockchain-based voting systems are a newer development. Blockchain provides secure, decentralized, and tamper-proof storage of votes. It removes the need for third-party involvement and enhances transparency and trust. Platforms like Votem and Voatz use blockchain technology to ensure encrypted ballots and secure audit trails.

Security is critical in e-voting systems. Techniques such as homomorphic encryption, blind signatures, and zero-knowledge proofs help protect voter privacy, ensure anonymity, and prevent fraud. However, challenges remain, including security threats, transparency concerns, and building public trust. For successful adoption, especially in developing countries and academic institutions, systems must be secure, transparent, easy to use, and adapted to local needs.

CHAPTER THREE

3.1 METHODOLOGY

The development of the Online Voting System followed the Waterfall Model, which involves sequential phases of system development. The process began with requirement analysis, where system objectives and features were identified. This was followed by system design, where database structures and user interfaces were planned.

During the implementation phase, the system was developed using PHP, Oracle Database, and frontend technologies such as HTML, CSS, and JavaScript. Testing was conducted to ensure system functionality, security, and reliability before deployment.

WATERFALL MODEL IMPLEMENTATION

The Online Voting System was developed using the **Waterfall Model**, which is a sequential software development lifecycle approach where each phase must be completed before moving to the next. Below is an explanation of how each phase was implemented specifically for this project:

PHASE 1: REQUIREMENT ANALYSIS

What We Did:

We began by studying the existing manual voting system at DIT to understand its problems. We interviewed potential users including students, lecturers, to identify what they needed from an online voting system.

Requirements Identified:

Category	Requirements Gathered
User Requirements	Simple registration, easy login, clear voting interface, ability to see results
Admin Requirements	Manage candidates, monitor voting, view results, generate reports
Security Requirements	Prevent duplicate voting, secure passwords, role-based access
Technical Requirements	Web-based platform, Oracle database, real-time results

Table 1.requirement for the system

PHASE 2: SYSTEM DESIGN

What We Did:

We created detailed designs for both the database and user interface based on the requirements gathered.

Database Design:

We designed three main tables:

- **USER table** - Stores voter and admin information
- **CANDIDATE table** - Stores candidate details
- **VOTE table** - Records all votes cast

Key Design Decisions:

- Used primary keys for unique identification
- Implemented foreign keys to link tables
- Added unique constraints to prevent duplicate voting
- Designed sequences and triggers for automatic ID generation

User Interface Design:

We designed wireframes for:

- Registration and login pages
- Admin dashboard for candidate management
- Voter dashboard for casting votes
- Results page for displaying outcomes

Output: ER diagrams, table structures, interface mockups, and system architecture design.

PHASE 3: IMPLEMENTATION

Backend Implementation (PHP):

- Created registration scripts to insert new users
- Developed login system with password verification
- Built admin functions to add/edit/delete candidates
- Implemented vote casting with duplicate checking
- Created result calculation queries

Frontend Implementation:

- HTML for page structure
- CSS for styling and responsive design
- JavaScript for form validation and interactivity

Output: A fully functional Online Voting System with all features working.

PHASE 4: TESTING**What We Did:**

We systematically tested every part of the system to ensure it worked correctly and securely.

Types of Testing Conducted:

Test Type	What We Tested	Results
Unit Testing	Individual functions (login, registration, vote casting)	All functions worked correctly
Integration Testing	How modules work together	Smooth data flow between frontend, backend, and database
Security Testing	Authentication, duplicate voting	System blocked all duplicate attempts, passwords were secure
Duplicate Voting Test	Trying to vote twice for same position	System rejected second attempt
User Acceptance	Real users tested the system	Positive feedback, easy to use

CHAPTER FOUR

4.1 SYSTEM DESIGN

The system uses a three-tier architecture consisting of the presentation layer, application layer, and database layer. The presentation layer provides the user interface, the application layer handles business logic, and the database layer stores system data securely.

4.1.1 SYSTEM ARCHITECTURE

The Online Voting System follows a three-tier architecture:

1. Presentation Layer – Provides the user interface through web pages.
2. Application Layer – Contains system logic implemented using PHP.
3. Database Layer – Stores and manages system data using Oracle Database.

4.1.2 DATABASE STRUCTURE:

The database design includes three main tables: Users, Candidates, and Votes. Relationships between these tables ensure proper data integrity and prevent duplicate voting through unique constraints. UML diagrams such as use case diagrams and entity relationship diagrams were used to model system interactions and structure.

The system database consists of three main tables:

1. Users Table

Stores information about all registered users including administrators and voters.

Fields include:

- User ID
- Full Name
- Username
- Password
- Role
- Registration Date

Attribute Name	Data Type	Data Length	Constraint	Key Type	Null Allowed	Description
id	NUMBER	–	PRIMARY KEY	Primary Key	No	Unique identifier for each user
full name	VARCHAR2	100	–	–	Yes	Full name of user
username	VARCHAR2	100	UNIQUE	–	No	Login username (must be unique)
password	VARCHAR2	255	–	–	No	Encrypted password
role	VARCHAR2	20	–	–	No	User role (Admin or Voter)
created at	DATE	–	DEFAULT SYSDATE	–	No	Account creation timestamp

Table 2.fully attributed table for user table.

2. Candidates Table

Stores information about election candidates such as:

- Candidate ID
- Candidate Name
- Position Contested
- Election Year
- Election Date
- Candidate Photo

Attribute name	Data type	Data length	Key type	Null allowed	description	
id	number	-	Primary key	No	Unique identifier for each candidate	
Name	VARCHAR2	100	-	No	Full name of candidate	
Position	VARCHAR2	50	-	No	Position being contested	
Election year	VARCHAR2	4	-	No	Year of election	
Election_date	DATE	-	-	No	Election day	
photo	VARCHAR2	255	-	yes	photo	
Created_at	DATE	-	Default (sysdate)	no	Record timestamp	

Table 3.Fully attributed table for candidate table.

3. Votes Table

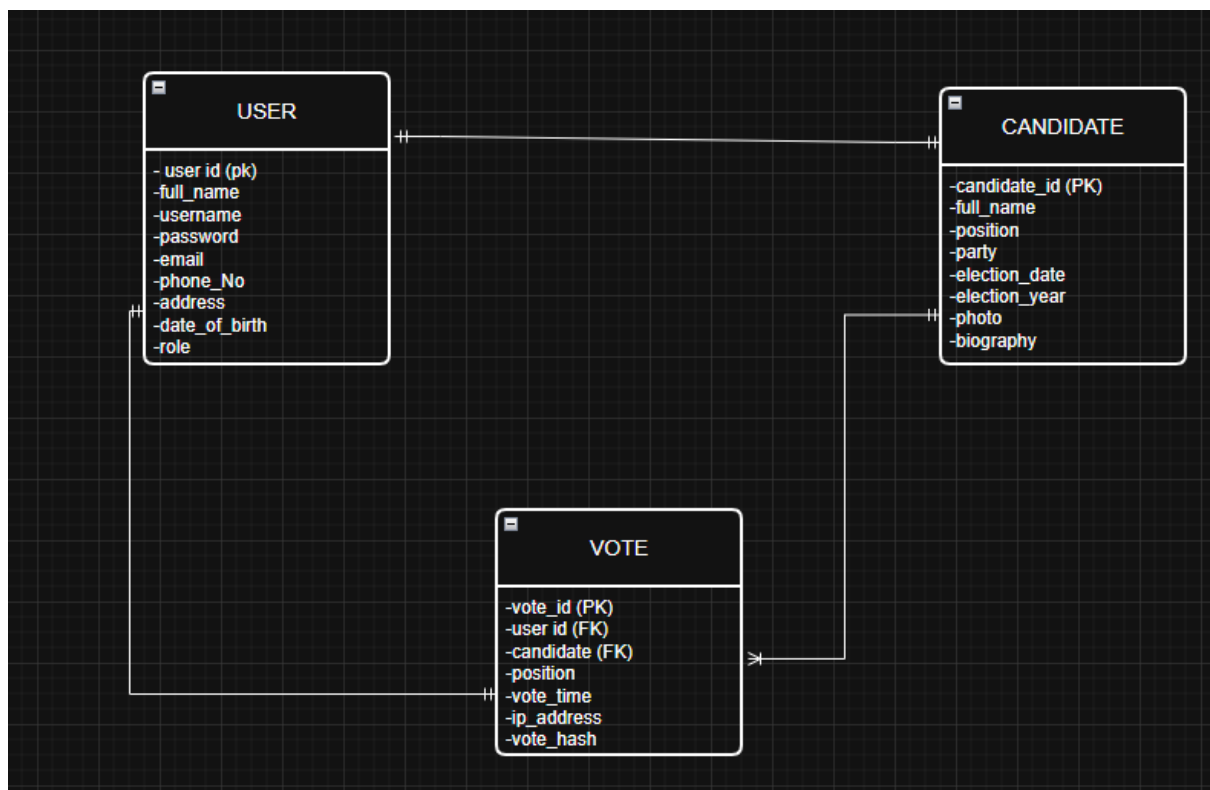
Stores voting records and ensures voting integrity. It contains:

- Vote ID
- User ID
- Candidate ID
- Position Voted
- Voting Time

Attribute Name	Data Type	Data Length	Null Allowed	Description
id	NUMBER	—	No	Unique identifier for each election
election_name	VARCHAR2	100	No	Name of the election
election_year	VARCHAR2	4	No	Year of the election
start_date	DATE	—	No	Voting start date
end_date	DATE	—	No	Voting end date
status	VARCHAR2	20	No	Current election status
created_at	DATE	—	No	Record creation timestamp

Table 4.fully attributed table for votes table

ERD DIAGRAM

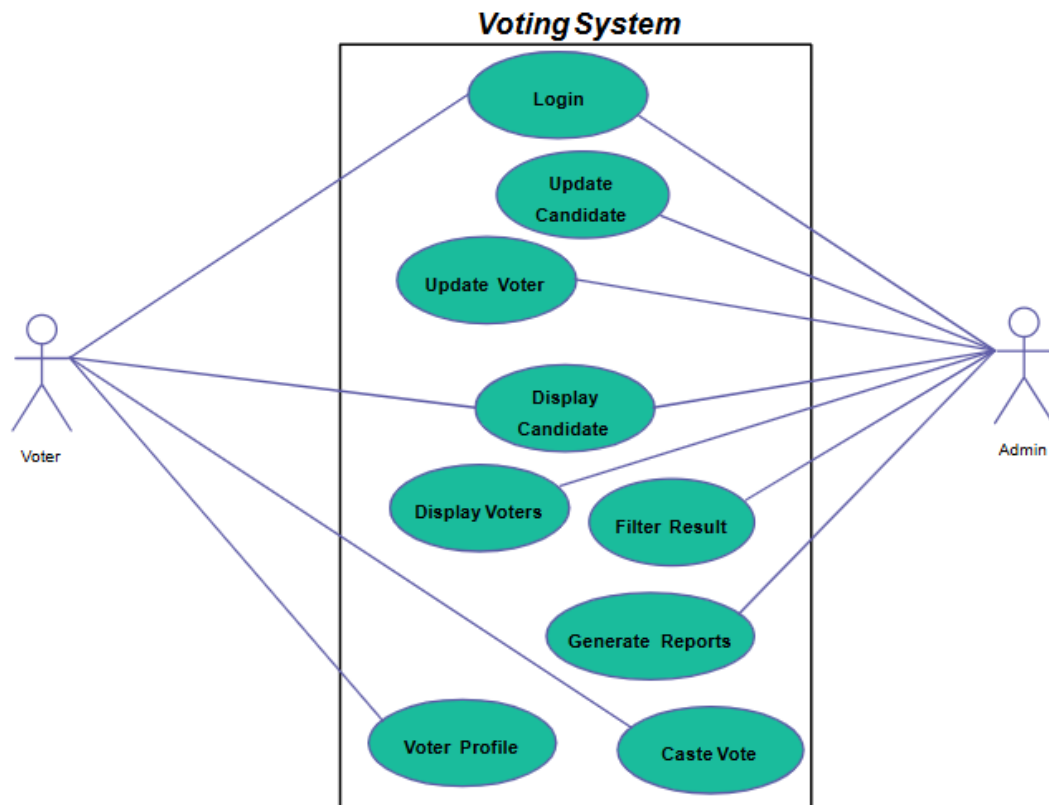


RELATIONSHIPS BETWEEN ENTITIES

The Online Voting System consists of three main entities connected through two primary relationships. The first relationship exists between USER and VOTE, which is a one-to-many relationship, meaning that one user can cast many votes across different positions, but each vote belongs to exactly one user. The second relationship exists between CANDIDATE and VOTE, which is also a one-to-many relationship, where one candidate can receive many votes from different users, but each vote is for exactly one candidate. These relationships are implemented through foreign keys in the VOTE table, where user_id references the USER table and candidate_id references the CANDIDATE table. This structure ensures data integrity by preventing orphaned records and maintaining referential integrity throughout the system. The VOTE table acts as an associative entity that links users to candidates, creating a complete trail of all voting activities while enforcing the business rule that each user can vote only once per position through additional unique constraints.

USE CASE DIAGRAM

1. USER CASE



ACTORS (IMPLIED)

Based on the use cases listed, the system has two main actors:

Actor	Description
Voter	Regular user who registers and casts votes
Administrator	System admin who manages the election process

USE CASES EXPLAINED

1. Login

- Actor: Both Voter and Admin
- Description: Entry point to the system
- Purpose: Authenticates users before allowing access to other functions
- Precondition: User must have registered account
- Postcondition: User gains access based on role (voter or admin privileges)

2. Update Candidate

- Actor: Administrator ONLY
- Description: Add, modify, or remove candidate information
- Functions include:
 - Adding new candidates
 - Editing candidate details (name, position, photo)
 - Removing candidates from election
- Database Table Affected: Candidates table

3. Update Voter

- Actor: Administrator ONLY
- Description: Manage voter registrations
- Functions include:
 - Approving/rejecting voter registrations
 - Editing voter information
 - Resetting voter passwords
 - Activating/deactivating voter accounts
- Database Table Affected: Users table

4. Display Candidate

- Actor: Both Voter and Admin
- Description: View list of candidates
- For Voters: See who they can vote for
- For Admins: Verify candidate list is correct and can add or delete them
- Database Table Affected: Candidates table

5. Display Voters

- Actor: Administrator ONLY
- Description: View registered voters list
- Purpose: Monitor voter registrations, check participation
- Database Table Affected: Users table (SELECT only)

6. Filter Result

- Actor: Both Voter and Admin
- Description: View election results with filtering options
- Filter options may include:
 - By position
 - By candidate
 - By date/ time
 - By department (if applicable)
- Database Table Affected: Votes table (aggregate queries)

7. Generate Reports

- Actor: Administrator ONLY
- Description: Create formal election reports
- Report types:
 - Final election results
 - Voter turnout statistics
 - Vote distribution by position
 - Audit trail reports
- Database Tables Affected: All tables (complex queries)

8. Voter Profile

- Actor: Voter ONLY
- Description: View and manage personal account
- Functions include:
 - View personal information
 - Change password
 - Update contact details
 - View voting history (if applicable)
- Database Table Affected: Users table

9. Cast Vote

- Actor: Voter ONLY
- Description: The core functionality - submitting a vote
- Process:
 1. Select position
 2. Choose candidate
 3. Confirm vote
 4. System records vote
 5. Prevention of duplicate voting
- Database Table Affected: Votes table (INSERT)
- Critical Feature: The system must verify user hasn't already voted for that position

TECHNOLOGY USED

FRONTEND TECHNOLOGIES

HTML is used to create the structure of all web pages including the registration form, login page, voting interface, and results display. It provides the basic skeleton with elements like input fields, buttons, tables, and forms that users interact with.

CSS is used to style the entire website and make it visually appealing. It handles colours, fonts, layouts, and responsive design so the system works properly on desktops, tablets, and mobile phones. CSS also adds hover effects and visual feedback for better user experience.

JavaScript is used to add interactivity and perform client-side validation. It checks form inputs before submission to ensure data is correct, displays confirmation dialogs when voting, and provides dynamic updates without reloading pages.

BACKEND TECHNOLOGIES

PHP is the core server-side programming language that handles all business logic. It processes user registrations by inserting data into the database, authenticates logins by verifying passwords, manages candidate additions and edits by administrators, records vote after checking for duplicates, and calculates results by querying the database. PHP acts as the bridge between the user interface and the database.

Apache or Nginx serves as the web server that receives user requests and delivers the appropriate web pages. It ensures the website is accessible to all users through their browsers.

DATABASE TECHNOLOGIES

Oracle Database is the relational database management system that stores all persistent data. It securely holds user information, candidate details, and every vote cast in the system. Oracle provides reliable data storage with transaction support and data integrity features.

SQL is the language used to communicate with the Oracle database. It performs all data operations including INSERT for adding new users and votes, SELECT for retrieving candidate lists and calculating results, UPDATE for modifying records, and DELETE for removing data. SQL queries are executed from PHP to interact with the database.

DEVELOPMENT TOOLS

Visual Studio Code serves as the integrated development environment where all code is written, edited, and debugged. It provides syntax highlighting, code completion, and extensions that make development faster and easier.

XAMPP or WAMP creates a local development environment by bundling Apache, PHP, and MySQL together. It allows the team to build and test the system on their own computers before deploying to a production server.

SYSTEM ARCHITECTURE

The Online Voting System follows a three-tier architecture:

1. Presentation Layer – Provides the user interface through web pages.
2. Application Layer – Contains system logic implemented using PHP.
3. Database Layer – Stores and manages system data using Oracle Database.

CHAPTER FIVE

IMPLEMENTATION

The system was implemented using PHP for backend logic and Oracle Database for data storage. Features such as secure login authentication, vote recording, candidate management, and results display were developed. The use of prepared SQL statements and password hashing ensured system security.

INTRODUCTION TP DATABASE IMPLEMENTATION

3 main things:

Creates tables → where data is stored

Creates sequences → automatic number generators

Creates triggers → automatically assign IDs

So basically, building the **backend database** for your voting system.

USERS TABLE

What this table stores

This table keeps **all registered users** in the voting system.

These include:

- Admins
- Normal voters

Creating the table

```
CREATE TABLE users (  
    id NUMBER PRIMARY KEY,  
    fullname VARCHAR2(100),  
    username VARCHAR2(100) UNIQUE,  
    password VARCHAR2(255),  
    role VARCHAR2(20),  
    created_at DATE DEFAULT SYSDATE  
);
```

Figure 2.create table for user

What each column means:

Column	Meaning
id	Unique number for each user
full name	User's full name
username	Login username (must be unique)
password	Encrypted password
role	Either "admin" or "user"
created at date	when user registered

PRIMARY KEY = cannot repeat

UNIQUE = prevents duplicate usernames

DEFAULT SYSDATE = automatically saves current date

Sequence for Users

```
CREATE SEQUENCE seq_users START WITH 1;
```

Figure 3.create sequence.

What is a Sequence?

A sequence is an **automatic number generator**.

Instead of manually writing IDs like:

1, 2, 3, 4...

Oracle automatically generates them.

Trigger for Users

```
CREATE OR REPLACE TRIGGER trig_users_id  
BEFORE INSERT ON users  
FOR EACH ROW  
BEGIN  
SELECT seq_users.NEXTVAL INTO :NEW.id FROM dual;  
END;
```

Figure 4.triggers for trig_user

What is this doing?

This trigger runs automatically before inserting a user.

It:

- Gets next number from sequence
- Assigns it to the ID column

CANDIDATES TABLE

What it stores

This table keeps **all candidates** in the election.

Table Creation

```
CREATE TABLE candidates (  
  id NUMBER PRIMARY KEY,  
  name VARCHAR2(100),  
  position VARCHAR2(50),  
  election_year VARCHAR2(4),  
  election_date DATE,  
  photo VARCHAR2(255),  
  created_at DATE DEFAULT SYSDATE  
);
```

Figure 5.create table for candidates.

Columns explained:

Column	Meaning
id	Candidate ID
name	Candidate name
position	Position contesting
Election _year	Election year
election _date	Election date
photo	Candidate image path
Created _at	Date added

Sequence

```
CREATE SEQUENCE seq_candidates START WITH 1;
```

Trigger

```
CREATE OR REPLACE TRIGGER trig_candidates_id  
BEFORE INSERT ON candidates  
FOR EACH ROW  
BEGIN  
SELECT seq_candidates.NEXTVAL INTO :NEW.id FROM dual;  
END;
```

Figure 6. triggers for candidate.

Oracle automatically assigns candidate ID.

This ensures:

- No duplicate IDs
- Automatic numbering

VOTES TABLE

What this table does

This is the **most important table**.

It stores **actual votes cast by users**.

Table Creation

```
CREATE TABLE elections (  
  
    id NUMBER PRIMARY KEY,  
    election_name VARCHAR2(100),  
    election_year VARCHAR2(4),  
    start_date DATE,  
    end_date DATE,  
    status VARCHAR2(20),  
    created_at DATE DEFAULT SYSDATE  
  
);
```

Figure 7.create table for votes.

Column Explanation

Column	Meaning
id	Unique vote ID
User id	Who voted
Candidate id	Who they voted for
position	Position voted
Vote time	When vote happened

FOREIGN KEYS

FOREIGN KEY (user _id) REFERENCES users(id)

Means:

The user must exist in the user's table.

FOREIGN KEY (candidate _id) REFERENCES candidates(id)

Means:

Candidate must exist before voting.

This prevents **invalid data**.

UNIQUE Constraint

UNIQUE(user_id, position)

This prevents:

A user voting twice for same position.

Example:

User 5 votes President → OK

User 5 votes President again → ERROR

This is how a system prevents **duplicate voting**.

Sequence

```
CREATE SEQUENCE seq_votes START WITH 1;
```

Trigger for Votes

```
CREATE OR REPLACE TRIGGER trig_votes_id
BEFORE INSERT ON votes
FOR EACH ROW
BEGIN
SELECT seq_votes.NEXTVAL INTO :NEW.id FROM dual;
END;
```

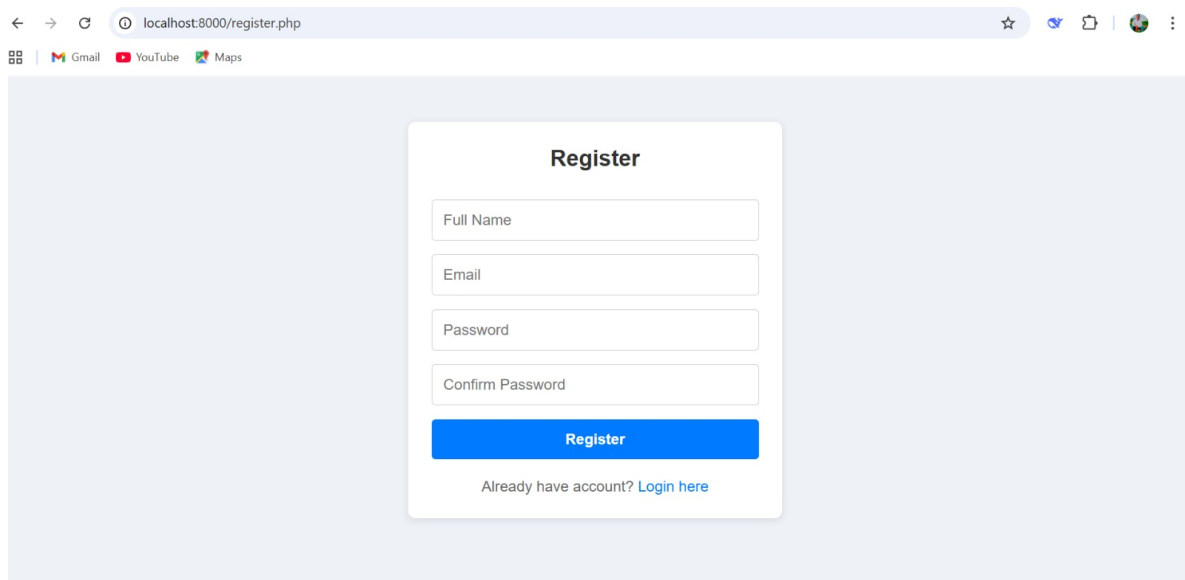
Figure 8.trigger for votes

Automatically generates vote ID.

INTRODUCTION OF SYSTEM IMPLEMENTATION

REGISTRATION PAGE

This page allows new users to create accounts in the system in order to collect user information and create new voter accounts. The system stores this data securely in the database with encrypted passwords.



The screenshot shows a web browser window with the address bar displaying 'localhost8000/register.php'. The browser's tab bar shows icons for Gmail, YouTube, and Maps. The main content area features a registration form titled 'Register'. The form contains four text input fields labeled 'Full Name', 'Email', 'Password', and 'Confirm Password'. Below these fields is a prominent blue button labeled 'Register'. At the bottom of the form, there is a text link that reads 'Already have account? Login here'.

Figure 9.registration page

LOGIN PAGE

This is the entry point to the Online Voting System where users authenticate themselves. In order to verify user identity and grant access to the appropriate dashboard based on their role (admin or regular voter). Only authenticated users can access the voting system

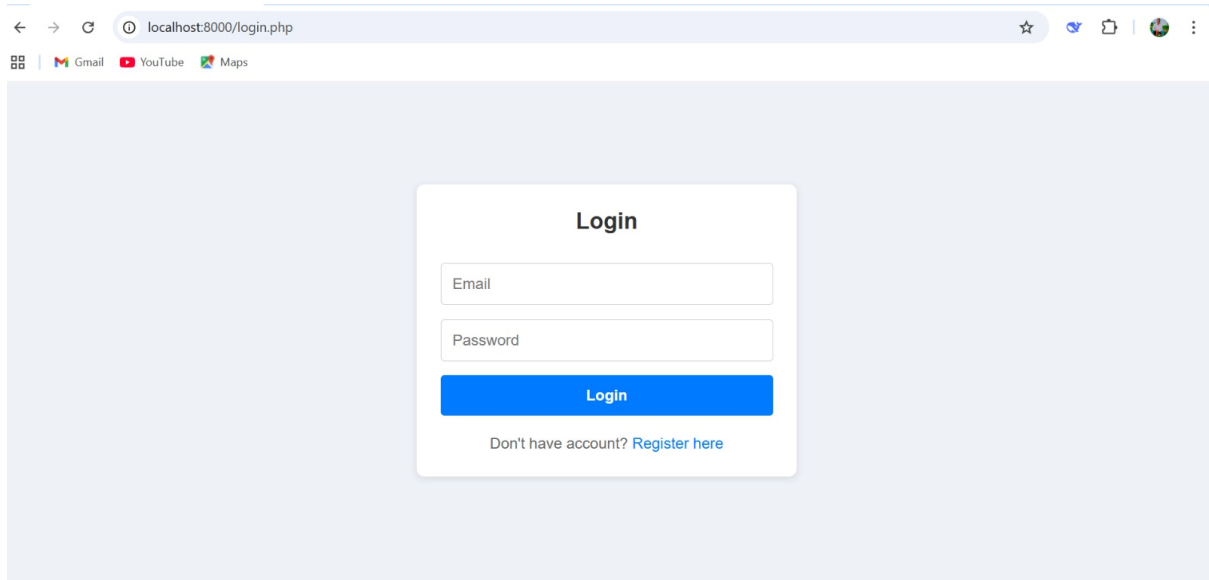
A screenshot of a web browser showing the login page at localhost:8000/login.php. The page has a light blue background. In the center, there is a white box with the title "Login". Inside this box, there are two input fields: "Email" and "Password". Below these fields is a blue button labeled "Login". At the bottom of the box, there is a link that says "Don't have account? Register here". The browser's address bar shows the URL, and the top of the browser has tabs for Gmail, YouTube, and Maps.

Figure 10. login page

VOTER DASHBOARD BEFORE VOTING

It shows both president candidates and vice president candidates. In this dashboard users can cast their votes but they can only choose one person in both positions.

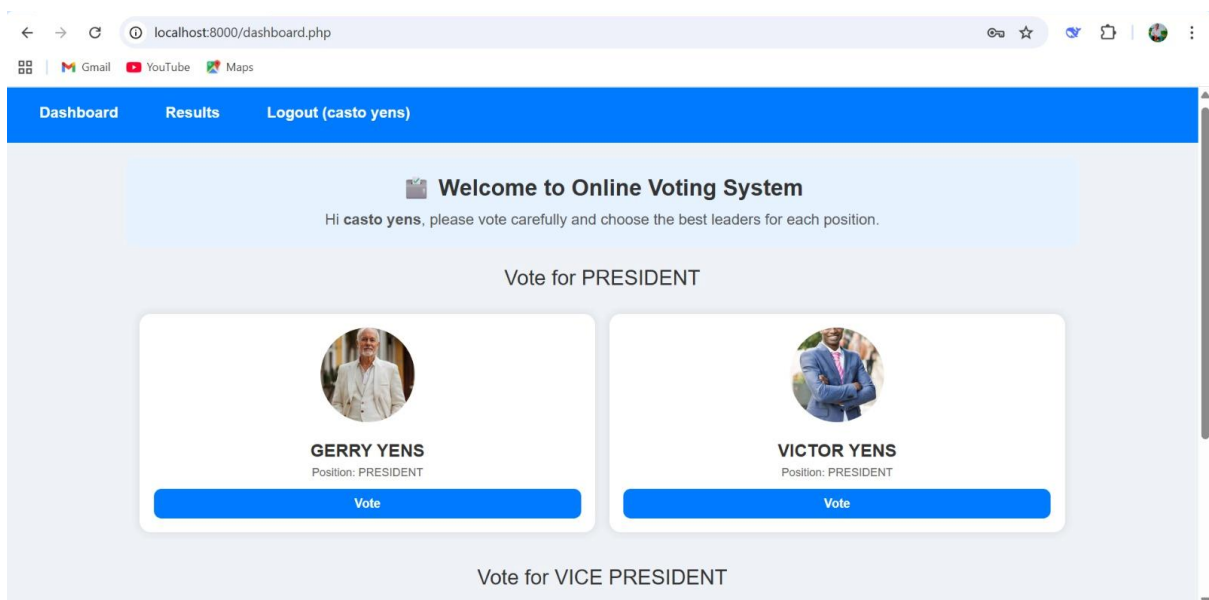
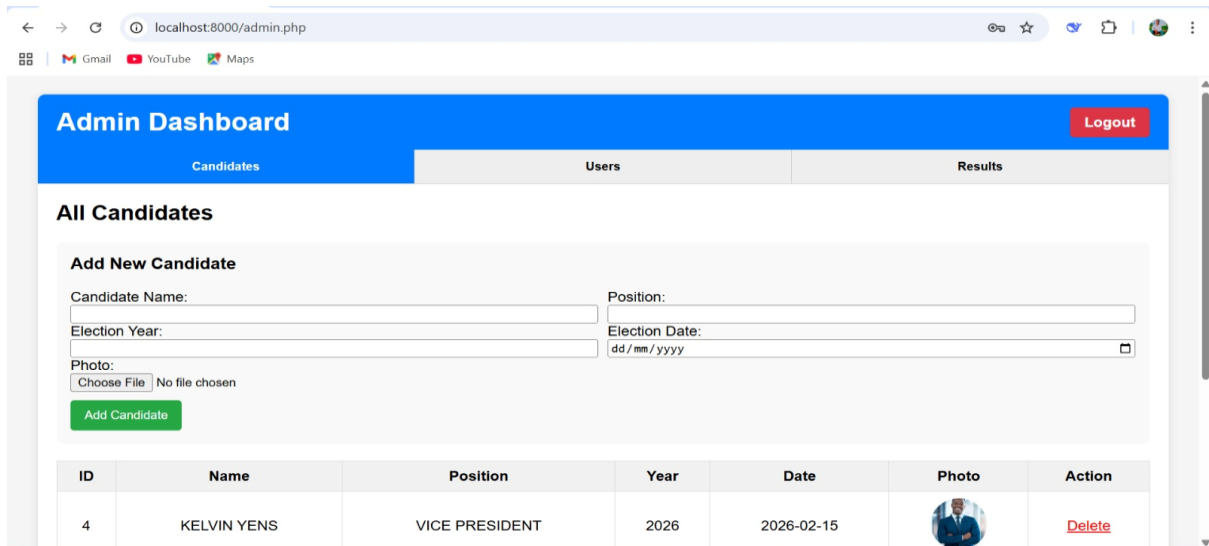
A screenshot of a web browser showing the voter dashboard at localhost:8000/dashboard.php. The page has a blue header with navigation links: "Dashboard", "Results", and "Logout (casto yens)". Below the header, there is a light blue box with the text "Welcome to Online Voting System" and "Hi casto yens, please vote carefully and choose the best leaders for each position.". The main content area is titled "Vote for PRESIDENT". It contains two white boxes, each with a circular profile picture of a candidate. The first box is for "GERRY YENS" with the position "PRESIDENT" and a blue "Vote" button. The second box is for "VICTOR YENS" with the position "PRESIDENT" and a blue "Vote" button. Below these boxes, the text "Vote for VICE PRESIDENT" is visible. The browser's address bar shows the URL, and the top of the browser has tabs for Gmail, YouTube, and Maps.

Figure 11. voters Dashboard.

ADMIN DASHBOARD

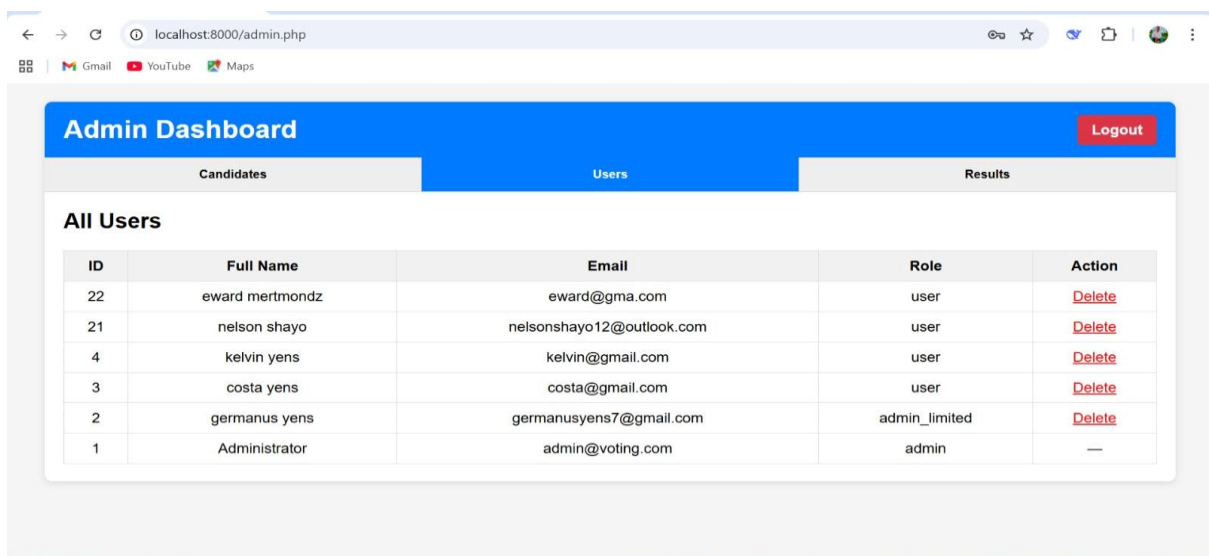
Admin has the privileges to add candidates, delete them, remove user also to compile the results.



The screenshot shows the 'Admin Dashboard' with a blue header bar containing the title and a 'Logout' button. Below the header is a navigation bar with three tabs: 'Candidates' (active), 'Users', and 'Results'. The main content area is titled 'All Candidates' and contains a form to 'Add New Candidate'. The form has fields for 'Candidate Name', 'Position', 'Election Year', 'Election Date', and 'Photo'. Below the form is a table listing candidates.

ID	Name	Position	Year	Date	Photo	Action
4	KELVIN YENS	VICE PRESIDENT	2026	2026-02-15		Delete

Figure 12.admin dashboard page_1.



The screenshot shows the 'Admin Dashboard' with a blue header bar containing the title and a 'Logout' button. Below the header is a navigation bar with three tabs: 'Candidates', 'Users' (active), and 'Results'. The main content area is titled 'All Users' and contains a table listing users.

ID	Full Name	Email	Role	Action
22	eward mertmondz	eward@gma.com	user	Delete
21	nelson shayo	nelsonshayo12@outlook.com	user	Delete
4	kelvin yens	kelvin@gmail.com	user	Delete
3	costa yens	costa@gmail.com	user	Delete
2	germanus yens	germanusyens7@gmail.com	admin_limited	Delete
1	Administrator	admin@voting.com	admin	—

Figure 13.admin dashboard page_2.

localhost:8000/admin.php

Admin Dashboard

Logout

Candidates Users Results

Election Results

PRESIDENT

Name	Votes	Photo
VICTOR YENS	3	
GERRY YENS	2	

VICE PRESIDENT

Name	Votes	Photo
COSTA YENS	3	

Figure 14.admin dashboard page_3.

LIMITED ADMIN DASHBOARD

Limited admin has no privileges of deleting candidates nor users but he or she can add candidates

The screenshot shows the 'All Candidates' page of the Admin Dashboard. The page has a blue header with 'Admin Dashboard' and a 'Logout' button. Below the header are three tabs: 'Candidates' (active), 'Users', and 'Results'. The main content area is titled 'All Candidates' and contains a form with fields for 'Candidate Name', 'Position', 'Year', and 'Date' (with a date picker set to 'dd/mm/yyyy'). There is a 'Choose File' button for a photo and an 'Add Candidate' button. Below the form is a table with the following data:

ID	Name	Position	Year	Date	Photo	Action
4	KELVIN YENS	VICE PRESIDENT	2026	2026-02-15		—
3	COSTA YENS	VICE PRESIDENT	2026	2026-02-15		—
2	GERRY YENS	PRESIDENT	2026	2026-02-15		—

Figure 15.limited admin dashboard page_1.

The screenshot shows the 'All Users' page of the Admin Dashboard. The page has a blue header with 'Admin Dashboard' and a 'Logout' button. Below the header are three tabs: 'Candidates', 'Users' (active), and 'Results'. The main content area is titled 'All Users' and contains a table with the following data:

ID	Name	Email	Role	Action
41	casto yens	casto@gmail.com	user	—
22	eward mertmondz	eward@gma.com	user	—
21	nelson shayo	nelsonshayo12@outlook.com	user	—
4	kelvin yens	kelvin@gmail.com	user	—
3	costa yens	costa@gmail.com	user	—
2	germanus yens	germanusyens7@gmail.com	admin_limited	—
1	Administrator	admin@voting.com	admin	—

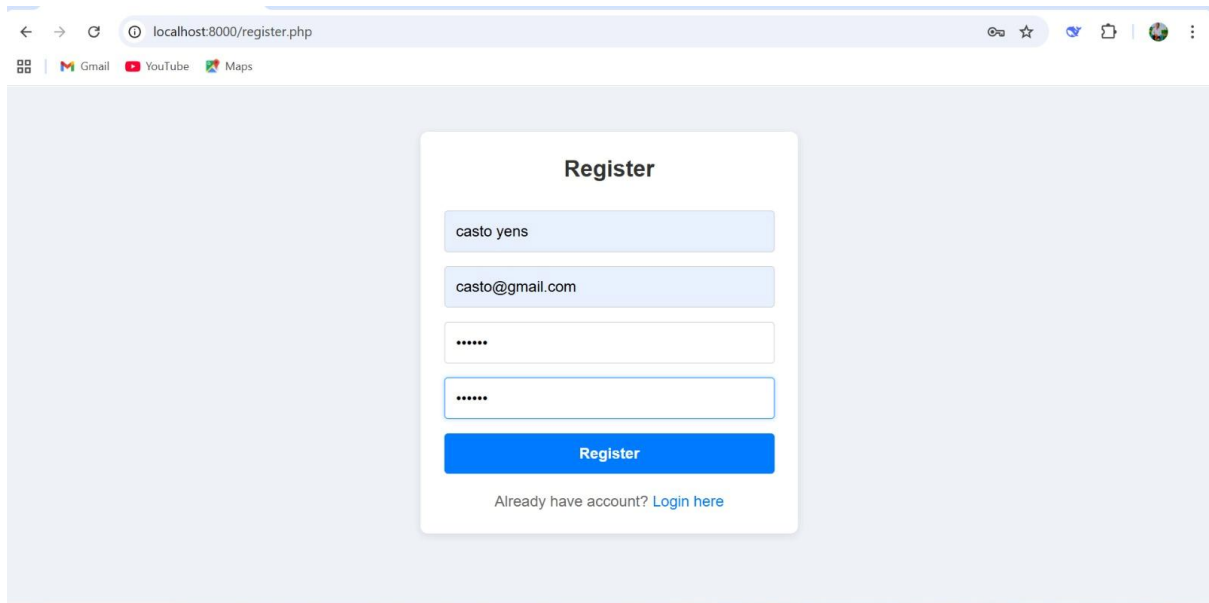
Figure 16.limited admin dashboard page_2.

CHAPTER FIVE

5.1 TESTING AND RESULTS

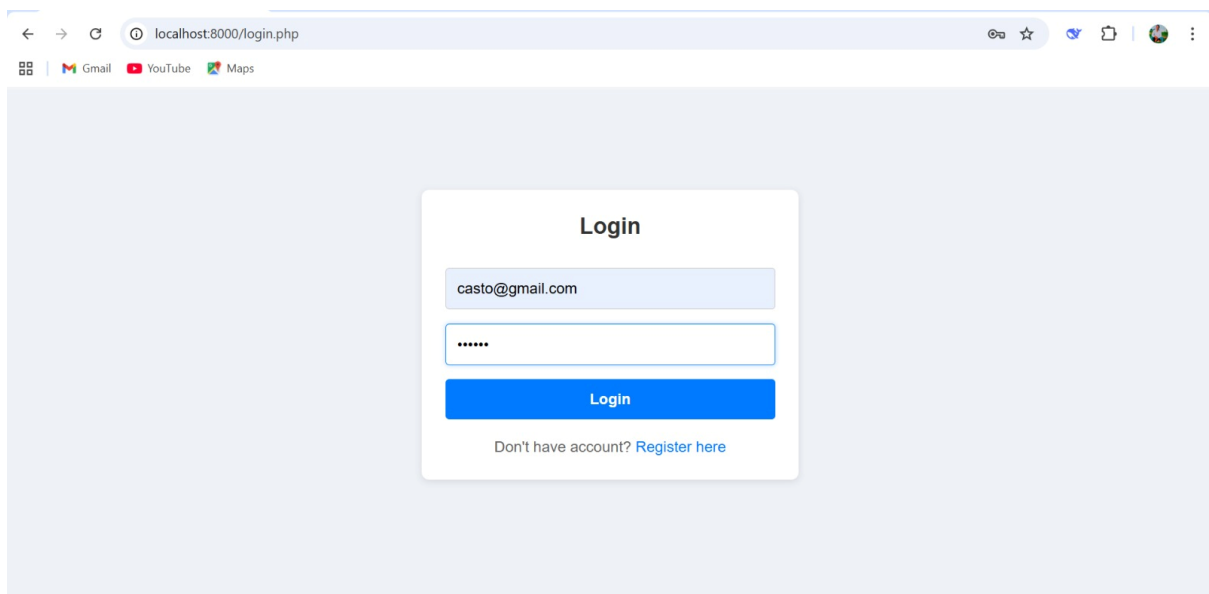
The system underwent various testing methods including unit testing, integration testing, and user acceptance testing. Results showed that the system successfully prevented duplicate voting, maintained data integrity, and provided accurate real-time results. Performance testing indicated fast response times and reliable operation.

Firstly, we are going to register a user name casto yens



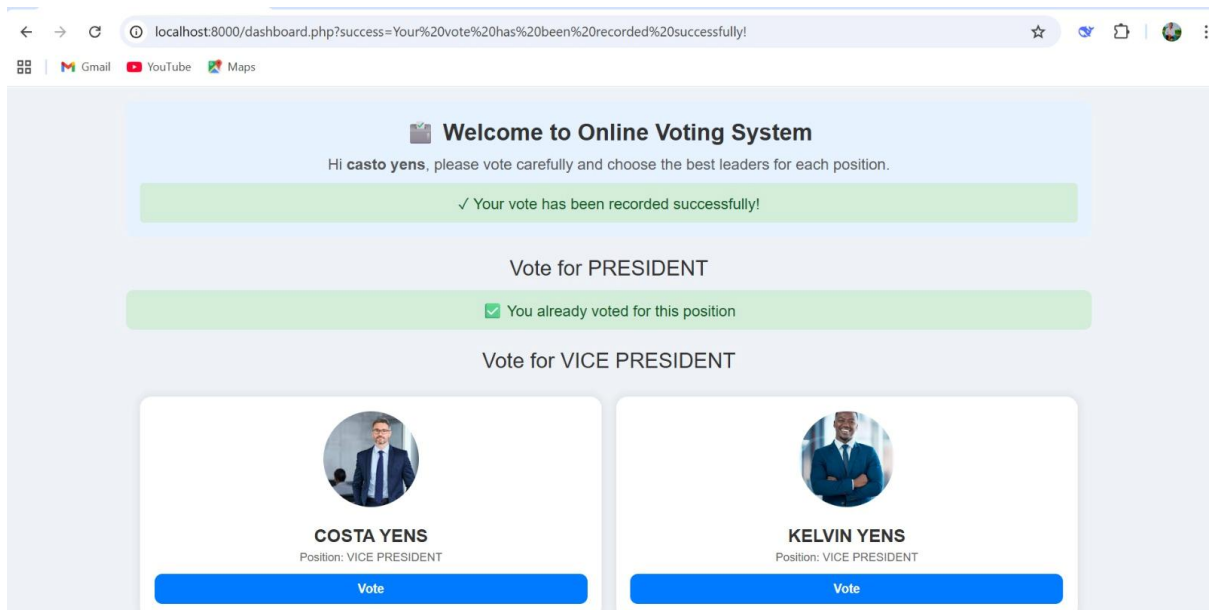
A screenshot of a web browser showing a registration form titled "Register". The form is centered on a light blue background. It contains four input fields: a text field with "casto yens", an email field with "casto@gmail.com", and two password fields, both masked with "*****". Below the fields is a blue "Register" button. At the bottom of the form, there is a link: "Already have account? [Login here](#)". The browser's address bar shows "localhost:8000/register.php".

After registering he is going to login in his account



A screenshot of a web browser showing a login form titled "Login". The form is centered on a light blue background. It contains two input fields: an email field with "casto@gmail.com" and a password field masked with "*****". Below the fields is a blue "Login" button. At the bottom of the form, there is a link: "Don't have account? [Register here](#)". The browser's address bar shows "localhost:8000/login.php".

From there he is going to enter into the voting dashboard and vote



After voting he can't vote again if he login again

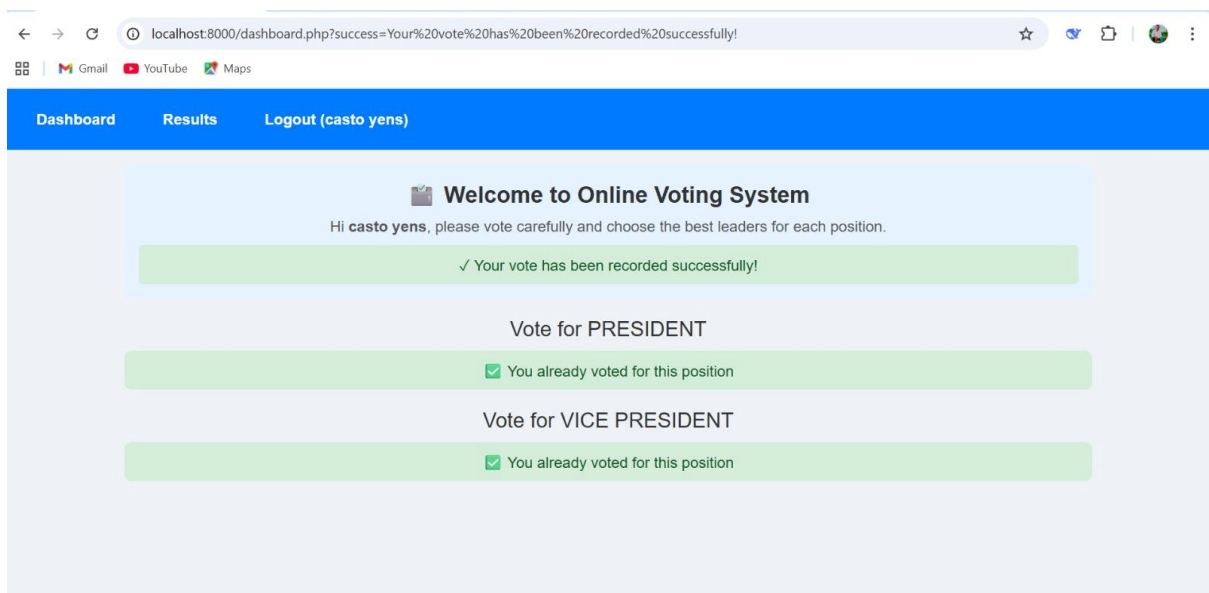


Figure 17. testing for multiple voting.

5.2 CONCLUSION AND REMMENDATIONS

The Online Voting System developed in this project demonstrates how web technologies and database systems can be integrated to create a secure and efficient electronic election platform. The system successfully addresses challenges found in traditional paper-based voting, such as slow counting, high operational costs, and the risk of duplicate voting. By using PHP and Oracle Database, the platform provides secure user registration, authenticated login, role-based access control, candidate management, and real-time result display. Database features such as primary keys, foreign keys, sequences, triggers, and unique constraints were effectively implemented to ensure data integrity and prevent multiple voting.

Although the system performs reliably and meets its main objectives, future improvements can enhance its effectiveness. Additional security measures such as two-factor authentication and vote encryption can be introduced. Expanding the system to support mobile access and conducting large-scale performance testing would also improve usability and scalability. Overall, the project provides a strong foundation for modern electronic voting solutions

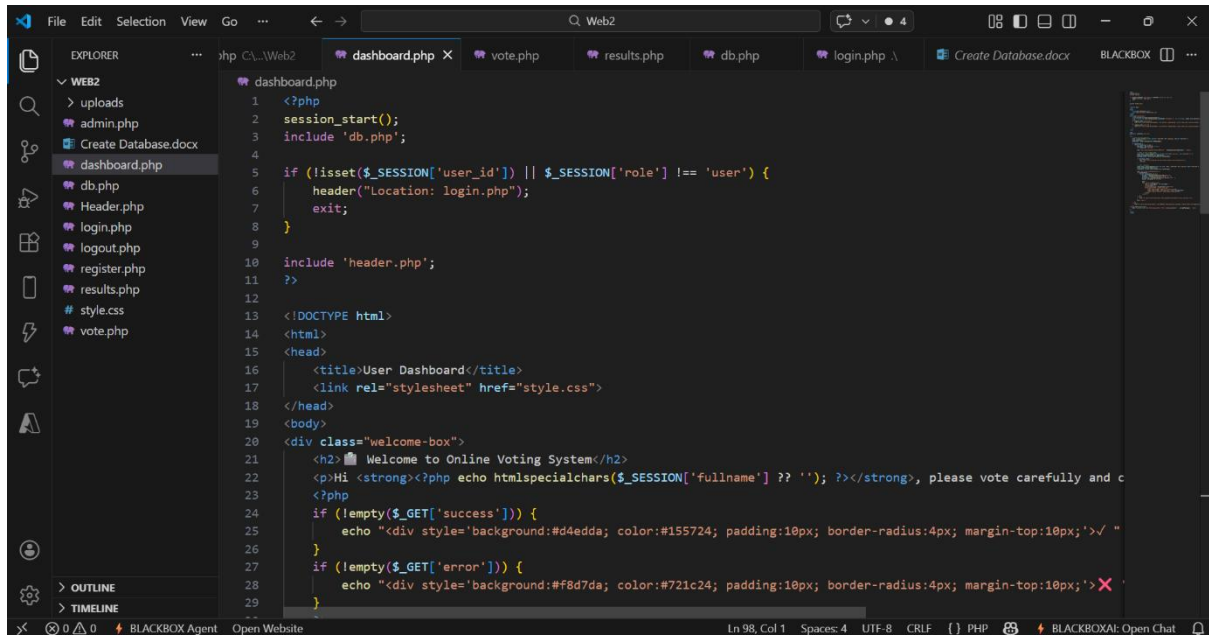
REFERENCES

Elmasri, R., & Navathe, S. (2016). Fundamentals of Database Systems. Pearson.

Welling, L., & Thomson, L. (2017). PHP and MySQL Web Development. Addison-Wesley.

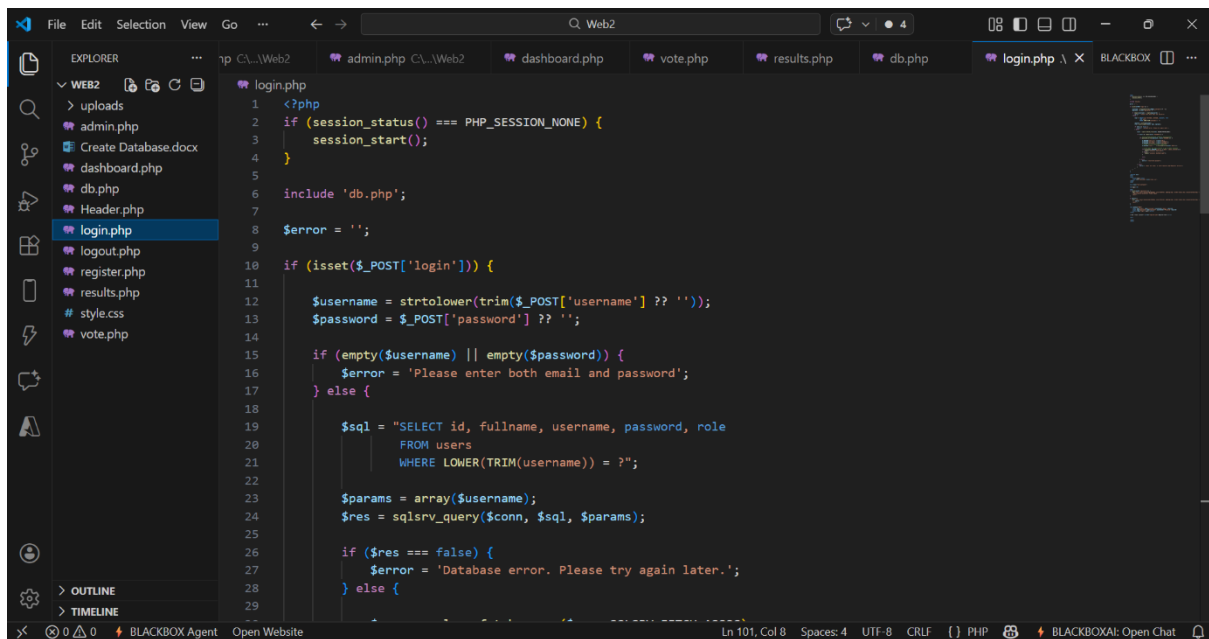
Oracle Corporation. (2023). Oracle Database SQL Language Reference.

APPENDICES



```
1 <?php
2 session_start();
3 include 'db.php';
4
5 if (!isset($_SESSION['user_id']) || $_SESSION['role'] !== 'user') {
6     header("Location: login.php");
7     exit;
8 }
9
10 include 'header.php';
11 ?>
12
13 <(DOCTYPE html>
14 <html>
15 <head>
16     <title>User Dashboard</title>
17     <link rel="stylesheet" href="style.css">
18 </head>
19 <body>
20 <div class="welcome-box">
21     <h2> Welcome to Online Voting System</h2>
22     <p>Hi <strong><?php echo htmlspecialchars($_SESSION['fullname']) ?? ''; ?></strong>, please vote carefully and c
23 <?php
24 if (empty($_GET['success'])) {
25     echo "<div style='background:#d4edda; color:#155724; padding:10px; border-radius:4px; margin-top:10px;'> "
26 }
27 if (empty($_GET['error'])) {
28     echo "<div style='background:#f8d7da; color:#721c24; padding:10px; border-radius:4px; margin-top:10px;'>X "
29 }
```

Figure 18.general code



```
1 <?php
2 if (session_status() === PHP_SESSION_NONE) {
3     session_start();
4 }
5
6 include 'db.php';
7
8 $error = '';
9
10 if (isset($_POST['login'])) {
11
12     $username = strtolower(trim($_POST['username']) ?? '');
13     $password = $_POST['password'] ?? '';
14
15     if (empty($username) || empty($password)) {
16         $error = 'Please enter both email and password';
17     } else {
18
19         $sql = "SELECT id, fullname, username, password, role
20                 FROM users
21                 WHERE LOWER(TRIM(username)) = ?";
22
23         $params = array($username);
24         $res = sqlsrv_query($conn, $sql, $params);
25
26         if ($res === false) {
27             $error = 'Database error. Please try again later.';
28         } else {
29
30             $row = sqlsrv_fetch_array($res, SQLSRV_FETCH_ASSOC);
31             $user_id = $row['id'];
32             $password_hash = $row['password'];
33             $role = $row['role'];
34
35             if (password_verify($password, $password_hash)) {
36                 $_SESSION['user_id'] = $user_id;
37                 $_SESSION['role'] = $role;
38                 header("Location: dashboard.php");
39                 exit;
40             } else {
41                 $error = 'Invalid username or password';
42             }
43         }
44     }
45 }
```

Figure 19.backend code